

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Департамент программной
инженерии ФКН НИУ ВШЭ
Научный руководитель,
кандидат технических наук,
доцент департамента
программной инженерии
факультета компьютерных наук

_____ / Н.С. Белова /
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.02-06 51 01–1 ЛУ

Исполнители:
студент группы БПИ233

_____ / Б.А. Багавиев /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.02-06 51 01–1 ЛУ

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Программа и методика испытаний

RU.17701729.02-06 51 01–1

Листов 25

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Программа и методика испытаний» разработан в соответствии с ГОСТ 19.301–79 и содержит программу и методику проведения испытаний программного изделия «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (BunnyMQ).

Документ определяет объект и цель испытаний, устанавливает требования к программе, подлежащие проверке, описывает средства и порядок проведения испытаний, а также методы испытаний по каждому проверяемому показателю: модульное тестирование подсистемы хранения и FSM Raft, интеграционное тестирование кластера, бенчмарки дисковой подсистемы, проверка API с помощью `bunnymq-cli`.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101–77: Виды программ и программных документов.
2. ГОСТ 19.103–77: Обозначения программ и программных документов.
3. ГОСТ 19.104–78: Основные надписи.
4. ГОСТ 19.105–78: Общие требования к программным документам.
5. ГОСТ 19.106–78: Требования к программным документам, выполненным печатным способом.
6. ГОСТ 19.301–79: Программа и методика испытаний. Требования к содержанию и оформлению.
7. ГОСТ 19.603–78: Общие правила внесения изменений.

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	1
1.1. Наименование программы	1
1.2. Область применения	1
1.3. Состав программного изделия	1
2. ЦЕЛЬ ИСПЫТАНИЙ	3
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	4
3.1. Функциональные требования	4
3.1.1. Management API	4
3.1.2. Data API	4
3.1.3. Группы потребителей	5
3.1.4. Аутентификация	5
3.2. Требования к надёжности	5
3.3. Требования к производительности	5
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	6
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	7
5.1. Технические средства	7
5.2. Программные средства	7
5.3. Порядок проведения испытаний	7
6. МЕТОДЫ ИСПЫТАНИЙ	9
6.1. Статический анализ	9
6.1.1. Метод проведения	9
6.1.2. Проверяемые показатели	9
6.2. Бенчмарки дисковой подсистемы	9
6.2.1. Метод проведения	9
6.2.2. Проверяемые сценарии	10
6.2.3. Критерии успешности	10
6.3. Модульное тестирование Storage	11
6.3.1. Метод проведения	11
6.3.2. Проверяемые компоненты	11
6.3.3. Критерии успешности	13
6.4. Модульное тестирование FSM	13
6.4.1. Метод проведения	13
6.4.2. MetadataFSM	14
6.4.3. PartitionFSM	15
6.4.4. Критерии успешности	15
6.5. Интеграционное тестирование кластера	15
6.5.1. Метод проведения	15
6.5.2. Проверяемые сценарии	16

6.5.3. Критерии успешности	18
6.6. Функциональное тестирование CLI	18
6.6.1. Метод проведения	18
6.6.2. Порядок выполнения	18
6.6.3. Критерии успешности	18

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Объектом испытаний является программное изделие «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**» (далее — BunnyMQ).

Обозначение программы по ГОСТ 19.103-77: **RU.17701729.02-06**.

Вид программного изделия по ГОСТ 19.101-77: **компонент** — самостоятельный программный компонент, предназначенный для включения в состав программных систем.

1.2. Область применения

BunnyMQ предназначен для применения в инфраструктуре распределённых backend-приложений в качестве брокера сообщений. Система обеспечивает надёжную асинхронную передачу данных с сохранением порядка сообщений в пределах партии, репликацией на уровне каждой партии через алгоритм Raft и горизонтальным масштабированием за счёт партиционирования.

1.3. Состав программного изделия

Программное изделие включает следующие исполняемые компоненты и библиотеки:

1. `bunnymq` — серверный процесс брокера (`cmd/bunnymq`), запускается на каждом узле кластера; включает gRPC-серверы Data API (порт 9090) и Management API (порт 9091), HTTP-сервер метрик Prometheus (порт 9092);
2. `bunnymq-cli` — утилита командной строки для административных и отладочных операций (`cmd/bunnymq-cli`);
3. `pkg/client` — публичная Go-библиотека для встраивания в прикладной код производителей и потребителей.

Внутренние пакеты, охватываемые испытаниями:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- `internal/storage` — сегментированный append-only журнал (Storage, SegmentStorage, LogSegment, OffsetIndexSegment, TimeIndexSegment);
- `internal/metadata` — MetadataFSM (IStateMachine dragonboat, shard 0);
- `internal/partition` — PartitionFSM (IOOnDiskStateMachine dragonboat, shard 1...N);
- `internal/coordinator/cluster`, `internal/coordinator/data`, `internal/coordinator/group` — координаторы кластера, данных и групп потребителей;
- `internal/api/data`, `internal/api/management` — gRPC-серверы.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия разработанного программного изделия BunnyMQ требованиям, установленным в техническом задании (обозначение: RU.17701729.02-06 ТЗ 01–1).

В ходе испытаний проверяются:

1. выполнение функциональных требований к Management API и Data API (раздел 4.1 ТЗ): управление темами, публикация сообщений, чтение по смещению, работа с группами потребителей;
2. корректность работы подсистемы хранения (раздел 4.1 ТЗ): сегментирование журнала, индексирование по смещению и временной метке, политика хранения;
3. корректность восстановления после сбоя (раздел 4.2 ТЗ): torn write recovery через `applied.idx` и `Storage.TruncateTo`;
4. соответствие требованиям к производительности (раздел 4.8 ТЗ): пропускная способность записи, задержка при различных размерах пакетов и режимах синхронизации;
5. корректность работы кластера из трёх узлов (раздел 4.2 ТЗ): выборы лидера, терпимость к отказу одного узла, репликация данных;
6. корректность алгоритма Range Assignment для групп потребителей (раздел 4.1 ТЗ);
7. работоспособность `bunnymq-cli` для полного набора административных операций.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

Перечень требований, подлежащих проверке во время испытаний, определяется разделом 4 технического задания (RU.17701729.02-06 ТЗ 01–1).

3.1. Функциональные требования

3.1.1. Management API

1. `CreateTopic` — создание темы с заданным числом партиций, коэффициентом репликации и политикой хранения.
2. `DeleteTopic` — удаление темы и всех связанных данных.
3. `ListTopics` — получение перечня всех тем кластера.
4. `DescribeTopic` — получение метаданных темы: число партиций, RF, лидер каждой партиции.
5. `AlterTopicPartitions` — увеличение числа партиций темы.
6. `AlterTopicRetention` — изменение параметров политики хранения.
7. `DescribeCluster` — получение состояния кластера: перечень узлов с ролями LEADER / FOLLOWER.

3.1.2. Data API

1. `Produce с acks=0` — публикация пакета без ожидания подтверждения кворума.
2. `Produce с acks=all` — публикация пакета с ожиданием фиксации на кворуме Raft.
3. `Fetch с max_wait_ms > 0` — чтение данных по смещению с долгим опросом при их отсутствии.
4. `GetOffsets(EarliestOffset)` — получение наиболее раннего доступного смещения.
5. `GetOffsets(LatestOffset)` — получение следующего назначаемого смещения.
6. `GetOffsets(ByTimestamp)` — получение первого смещения за заданную временную метку.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.1.3. Группы потребителей

1. `JoinGroup` — регистрация участника в группе потребителей.
2. `Heartbeat` — обновление временной метки активности участника.
3. `LeaveGroup` — явный выход участника из группы.
4. `CommitOffset` — фиксация смещения для партии в группе.
5. `FetchCommittedOffsets` — получение зафиксированных смещений.
6. Автоматическое исключение участника при отсутствии `heartbeat` дольше `session_timeout_ms`.

3.1.4. Аутентификация

1. При непустом списке `auth.tokens` запросы без заголовка `bunmq-auth-token` или с неверным токеном отклоняются с кодом `UNAUTHENTICATED`.
2. При пустом списке `auth.tokens` (PLAINTEXT режим) все запросы принимаются.

3.2. Требования к надёжности

1. Torn write recovery: после имитации сбоя процесса в ходе записи пакета узел восстанавливается без потери ранее подтверждённых данных.
2. Graceful shutdown: процесс завершается корректно при получении `SIGTERM` без повреждения данных.
3. Кворум: при отказе одного из трёх узлов кластера запись и чтение продолжают на оставшихся узлах.

3.3. Требования к производительности

1. Пропускная способность записи при `acks=0`, пакет 65 536 байт: не менее 200 МБ/с на SSD-диске.
2. Задержка синхронной операции `fsync` для пакета 4 096 байт: не более 5 мс.
3. Задержка `Produce` с `acks=all` (p99, LAN с $RTT \leq 1$ мс): не более 50 мс.
4. Задержка `Fetch` при наличии данных (p99): не более 10 мс.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытания предъявляется следующий состав программной документации:

1. Техническое задание (ГОСТ 19.201–78) — обозначение:
RU.17701729.02-06 ТЗ 01–1.
2. Текст программы (ГОСТ 19.401–78) — обозначение:
RU.17701729.02-06 12 01–1.
3. Программа и методика испытаний (ГОСТ 19.301–79) — обозначение:
RU.17701729.02-06 51 01–1 (настоящий документ).
4. Пояснительная записка (ГОСТ 19.404–79) — обозначение:
RU.17701729.02-06 81 01–1.
5. Руководство оператора (ГОСТ 19.505–79) — обозначение:
RU.17701729.02-06 34 01–1.

Документация оформлена в соответствии с ГОСТ 19.106–78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Испытания проводятся на оборудовании со следующими характеристиками:

Таблица 5.1 — Технические средства проведения испытаний

Параметр	Значение
Процессор	Apple M2 Pro, 10 ядер (6 производительности + 4 эффективности)
Оперативная память	16 ГБ LPDDR5
Постоянное хранилище	SSD NVMe 512 ГБ (Apple APFS)
Операционная система	macOS 14 Sonoma (arm64)
Go	версия 1.25 (darwin/arm64)
Сетевой интерфейс	Loopback (lo0) для локального кластера из 3 узлов

5.2. Программные средства

Таблица 5.2 — Программные средства проведения испытаний

Средство	Назначение
go test	Встроенный фреймворк тестирования Go: модульные и интеграционные тесты
go test -bench	Бенчмарки с измерением ns/op, B/op, allocs/op
go test -race	Детектор гонок данных (race detector)
go test -count	Многократный запуск тестов для проверки воспроизводимости
bunnymq-cli	Функциональное тестирование API Management и Data через командную строку
golangci-lint	Статический анализ кода (errcheck, staticcheck, govet, revive)

5.3. Порядок проведения испытаний

Испытания проводятся в следующем порядке:

1. **Статический анализ** — проверка кода линтером перед запуском тестов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Бенчмарки дисковой подсистемы** — измерение характеристик дисковой записи для выбора оптимальных параметров открытия файла журнала. Выполняются первыми, так как их результаты влияют на настройку Storage.
3. **Модульное тестирование Storage** — проверка корректности работы каждого уровня иерархии хранения: LogSegment, OffsetIndexSegment, TimeIndexSegment, SegmentStorage, Storage. Выполняется изолированно с созданием временных каталогов.
4. **Модульное тестирование FSM** — проверка MetadataFSM и PartitionFSM: применение команд, сериализация/десериализация состояния, torn write recovery.
5. **Интеграционное тестирование кластера** — запуск кластера из 3 узлов в рамках одного тестового процесса (три NodeHost на loopback), проверка полного API: Management + Data + Consumer Groups. Выполняется последним, так как требует максимального времени (до 60 с на старт кластера и выборы лидера).
6. **Функциональное тестирование CLI** — ручная проверка `bunnymq-cli` по перечню команд (Приложение 2).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Статический анализ

6.1.1. Метод проведения

Статический анализ выполняется автоматически инструментом `golangci-lint` перед запуском тестов. Конфигурация линтера задаётся файлом `.golangci.yml` в корне репозитория.

Команда запуска:

```
golangci-lint run ./...
```

6.1.2. Проверяемые показатели

Таблица 6.1 — Линтеры и проверяемые свойства

Линтер	Проверяемое свойство
<code>errcheck</code>	Все возвращаемые ошибки должны быть явно обработаны
<code>staticcheck</code>	Статический анализ: недостижимый код, устаревший API, неверные форматные строки
<code>govet</code>	Подозрительные конструкции: неправильное использование <code>sync/atomic</code> , <code>Printf</code> с неверным типом
<code>revive</code>	Стиль кода: имена экспортируемых идентификаторов, отступы, комментарии

Результат испытания: отсутствие ошибок линтера (выход с кодом 0).

6.2. Бенчмарки дисковой подсистемы

6.2.1. Метод проведения

Бенчмарки реализованы в файле `benchmarks/disk_test.go` и запускаются стандартным инструментом `go test -bench`. Каждый бенчмарк измеряет скорость дисковой записи при различных размерах блока и режимах синхронизации.

Команда запуска:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
go test -bench=BenchmarkDiskWrite -benchmem -benchtime=5s ./
benchmarks/
```

6.2.2. Проверяемые сценарии

Таблица 6.2 — Сценарии бенчмарков дисковой записи

Имя	Размер блока	Флаги открытия	Описание
4096B_no_sync	4 096 байт	O_CREATE O_RDWR	Последовательная запись 4 КиБ без fsync
4096B_sync	4 096 байт	O_CREATE O_RDWR O_SYNC	Последовательная запись 4 КиБ с O_SYNC
128B_no_sync	128 байт	O_CREATE O_RDWR	Мелкие записи без fsync
128B_sync	128 байт	O_CREATE O_RDWR O_SYNC	Мелкие записи с O_SYNC
65536B_no_sync	65 536 байт	O_CREATE O_RDWR	Крупные записи без fsync
65536B_sync	65 536 байт	O_CREATE O_RDWR O_SYNC	Крупные записи с O_SYNC

Для каждого размера блока выполняются подбенчмарки:

- `sequential_sync` — запись с последующим явным `file.Sync()`;
- `sequential_no_sync` — запись без синхронизации;
- `random_sync` — случайная запись по адресу из диапазона 100×4 КиБ с `file.Sync()`;
- `random_no_sync` — случайная запись без синхронизации;
- `pure_sync` — только `file.Sync()` без записи.

6.2.3. Критерии успешности

Бенчмарк считается успешным, если:

- последовательная запись 65 536 байт без fsync выполняется со скоростью не менее 200 МБ/с;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- задержка `file.Sync()` для блока 4 096 байт не превышает 5 мс (5 000 000 нс/оп).

Результаты сохраняются и анализируются для выбора оптимальных флагов открытия `.log`-файла в `Storage`.

6.3. Модульное тестирование `Storage`

6.3.1. Метод проведения

Тесты реализованы в файле `internal/storage/log_segment_test.go` и смежных файлах `*_test.go`. Каждый тест создаёт временный каталог через вспомогательную функцию `testutil.TempDir` и удаляет его по завершении через `t.Cleanup`.

Команда запуска:

```
go test -v -count=1 -race ./internal/storage/...
```

Флаг `-race` позволяет обнаруживать гонки данных при конкурентном доступе к `Storage` (метод `Read` вызывается параллельно с `Append`).

6.3.2. Проверяемые компоненты

6.3.2.1. *LogSegment*

- `TestLogSegment_WriteRead` — запись произвольного пакета и последующее чтение полного содержимого файла.
- `TestLogSegment_Size` — проверка корректности счётчика `logSize` после серии записей.
- `TestLogSegment_OpenExisting` — открытие существующего файла: `logSize` инициализируется из `os.Stat`.
- `TestLogSegment_Seal` — запечатывание: файл переоткрывается в `O_RDONLY`, запись после запечатывания возвращает ошибку.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.3.2.2. *OffsetIndexSegment*

- `TestOffsetIndex_AppendLookup` — добавление N записей, бинарный поиск по каждому `relative_offset` возвращает корректную `position`.
- `TestOffsetIndex_EntrySize` — размер каждой записи строго равен 8 байтам.
- `TestOffsetIndex_BinarySearch_NotFound` — для отсутствующего смещения возвращается запись с ближайшим меньшим смещением.
- `TestOffsetIndex_Seal` — после запечатывания: файл усечён до фактического размера, запись запрещена.

6.3.2.3. *TimeIndexSegment*

- `TestTimeIndex_AppendLookup` — добавление N записей, поиск по временной метке возвращает корректное `relative_offset`.
- `TestTimeIndex_EntrySize` — размер каждой записи строго равен 12 байтам.
- `TestTimeIndex_EarliestAfter` — для временной метки, предшествующей первой записи, возвращается первое смещение.

6.3.2.4. *SegmentStorage*

- `TestSegmentStorage_AppendRead` — запись пакета с известным `base_offset`, чтение до `maxBytes`, проверка возврата полного пакета.
- `TestSegmentStorage_ReadMaxBytes` — при `maxBytes` меньше размера одного пакета метод возвращает ровно один пакет.
- `TestSegmentStorage_IndexSampling` — запись N пакетов, проверка числа записей в индексе (не каждый пакет индексируется).
- `TestSegmentStorage_Seal_and_Read` — после запечатывания сегмент доступен для чтения.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.3.2.5. Storage (верхний уровень)

- `TestStorage_AppendRead_SingleSegment` — запись и чтение в пределах одного сегмента.
- `TestStorage_SegmentRoll` — запись до превышения `segment_max_bytes`: проверяется создание нового сегмента.
- `TestStorage_ReadAcrossSegments` — запрос смещения из запечатанного сегмента возвращает корректные данные.
- `TestStorage_NewDataCh` — после `Append` предыдущий канал закрыт, новый канал открыт.
- `TestStorage_LongPoll` — горутина ждёт на `NewDataCh`, основной поток выполняет `Append`, горутина разблокируется.
- `TestStorage_EarliestLatestOffset` — проверка значений до и после записей.
- `TestStorage_TruncateTo` — усечение журнала до заданного смещения, последующее чтение возвращает только оставшиеся данные.
- `TestStorage_Retention` — вызов `EnforceRetention` удаляет сегменты, нарушающие политику; активный сегмент не удаляется.
- `TestStorage_ReadByTime` — чтение по временной метке возвращает пакет с `base_timestamp ≥ timestampMs`.
- `TestStorage_ConcurrentReadWrite` — запись из одной горутины, параллельное чтение из нескольких: race detector не фиксирует нарушений.

6.3.3. Критерии успешности

Все тесты проходят (PASS). Race detector не обнаруживает гонок (–race выход с кодом 0).

6.4. Модульное тестирование FSM

6.4.1. Метод проведения

Тесты реализованы в файлах `internal/metadata/*_test.go` и `internal/partition/*_test.go`. `MetadataFSM` тестируется без запуска `dragonboat`: команды применяются вызовом `fsm.Update([]sm.Entry{...})` напрямую.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Команда запуска:

```
go test -v -count=1 ./internal/metadata/... ./internal/partition/...
```

6.4.2. MetadataFSM

- `TestMetadataFSM_CreateTopic` — создание темы: проверяется создание `TopicMeta` и N экземпляров `PartitionMeta`, инкремент `NextShardID`.
- `TestMetadataFSM_CreateTopic_AlreadyExists` — повторное создание темы возвращает код `AlreadyExists` в `sm.Result`.
- `TestMetadataFSM_DeleteTopic` — удаление темы и всех связанных `PartitionMeta`.
- `TestMetadataFSM_Determinism` — одинаковая последовательность команд, применённая к двум независимым экземплярам FSM, даёт идентичное состояние.
- `TestMetadataFSM_SnapshotRestore` — сериализация и восстановление состояния через `SaveSnapshot/RecoverFromSnapshot`: состояние идентично.
- `TestMetadataFSM_JoinLeaveGroup` — регистрация участника, проверка `Groups[groupID].Members`, выход участника, проверка удаления.
- `TestMetadataFSM_Rebalance` — после `RebalanceConsumerGroupCmd` с двумя участниками и тремя партициями: первый участник получает партиции 0–1, второй — партицию 2.

6.4.2.1. Проверка детерминизма *Range Assignment*

Для группы с M участниками и P партициями проверяются:

Таблица 6.3 — Тест-кейсы Range Assignment

Р	М	Тест	Ожидаемое распределение
3	2	<code>TestRange_3p_2m</code>	<code>member_0: [0,1]; member_1: [2]</code>
4	2	<code>TestRange_4p_2m</code>	<code>member_0: [0,1]; member_1: [2,3]</code>
4	3	<code>TestRange_4p_3m</code>	<code>member_0: [0,1]; member_1: [2]; member_2: [3]</code>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Р	М	Тест	Ожидаемое распределение
1	3	TestRange_1p_3m	member_0: [0]; member_1: []; member_2: []
6	3	TestRange_6p_3m	member_0: [0,1]; member_1: [2,3]; member_2: [4,5]

6.4.3. PartitionFSM

- TestPartitionFSM_Open_Empty — открытие пустого каталога: создаётся начальный сегмент, LatestOffset = 0.
- TestPartitionFSM_Update_AppendBatch — применение команды AppendBatch: пакет записывается в Storage, applied.idx обновляется.
- TestPartitionFSM_TornWriteRecovery — имитация сбоя: пакет записан в .log, но applied.idx не обновлён; при следующем Open лишние данные отбрасываются, LatestOffset совпадает с applied.idx.
- TestPartitionFSM_Lookup_Read — запрос через Lookup возвращает ранее записанные данные.

6.4.4. Критерии успешности

Все тесты проходят (PASS). Тест TestMetadataFSM_Determinism подтверждает идентичность состояний двух FSM.

6.5. Интеграционное тестирование кластера

6.5.1. Метод проведения

Интеграционные тесты поднимают кластер из трёх узлов dragonboat внутри одного тестового процесса, используя loopback-адреса (127.0.0.1:8081–8083 для Raft, 127.0.0.1:9091–9093 для Management API, 127.0.0.1:9090–9092 для Data API). Каждый тест создаёт независимый временный каталог. Тесты помечены тегом integration:

```
go test -v -count=1 -timeout 120s -tags=integration ./...
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.5.2. Проверяемые сценарии

6.5.2.1. Управление темами

- `TestCluster_CreateTopic` — создание темы через Management API, описание (`DescribeTopic`) подтверждает правильное число партиций и коэффициент репликации.
- `TestCluster_DeleteTopic` — удаление темы, последующий `DescribeTopic` возвращает `NOT_FOUND`.
- `TestCluster_AlterPartitions` — увеличение числа партиций, `DescribeTopic` отражает новое значение.
- `TestCluster_AlterRetention` — изменение `retention_ms`, последующее `DescribeTopic` отражает новое значение.

6.5.2.2. Публикация и чтение

- `TestCluster_ProduceFetch_acks0` — публикация 100 пакетов с `acks=0`, последующее `Fetch` по смещению 0 возвращает все 100 пакетов в порядке публикации.
- `TestCluster_ProduceFetch_acksAll` — публикация с `acks=all`, `Fetch` возвращает данные только после подтверждения кворума.
- `TestCluster_Fetch_LongPoll` — `Fetch` с `max_wait_ms=2000` при пустом журнале: клиент блокируется до появления данных, опубликованных через 500 мс в другой горутине.
- `TestCluster_GetOffsets` — после записи N пакетов: `EarliestOffset = 0`, `LatestOffset = N × record_count`, `ByTimestamp` возвращает корректное смещение.
- `TestCluster_NotLeader_Redirect` — запрос к не-лидеру возвращает `NOT_FOUND (NotLeader)` с адресом лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.5.2.3. Отказоустойчивость

- `TestCluster_NodeFailure_QuorumSurvives` — остановка одного из трёх узлов (`NodeHost.Close`); публикация и чтение продолжают работу на оставшихся двух узлах; после перезапуска отказавшего узла он воспроизводит Raft-журнал и синхронизирует данные.
- `TestCluster_LeaderElection` — принудительная остановка текущего лидера; в течение не более 5 секунд один из оставшихся узлов избирается лидером (`DescribeCluster` подтверждает роль LEADER).

6.5.2.4. Группы потребителей

- `TestCluster_ConsumerGroup_JoinHeartbeatLeave` — вступление, heartbeat, выход; `FetchCommittedOffsets` возвращает пустой результат после выхода.
- `TestCluster_ConsumerGroup_CommitFetch` — фиксация смещения 42 для партииции 0; `FetchCommittedOffsets` возвращает 42.
- `TestCluster_ConsumerGroup_Rebalance` — вступление двух участников в группу, подписанную на тему с 3 партициями; `JoinGroup` возвращает распределение по алгоритму Range Assignment (участник 0 получает партиции 0–1, участник 1 — партицию 2).
- `TestCluster_ConsumerGroup_SessionTimeout` — участник перестаёт отправлять heartbeat; через `session_timeout_ms` он автоматически исключается из группы.

6.5.2.5. Аутентификация

- `TestCluster_Auth_ValidToken` — запрос с корректным токеном обрабатывается.
- `TestCluster_Auth_InvalidToken` — запрос с неверным токеном возвращает UNAUTHENTICATED.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

— `TestCluster_Auth_Plaintext` — кластер с пустым списком токенов принимает запросы без заголовка.

6.5.3. Критерии успешности

Все тесты проходят (PASS). Тест `TestCluster_NodeFailure_QuorumSurvives` выполняется без возврата ошибок.

6.6. Функциональное тестирование CLI

6.6.1. Метод проведения

Функциональное тестирование `bunnumq-cli` выполняется вручную на запущенном локальном кластере из 3 узлов. Проверяется каждая команда из перечня (Приложение 2) путём сравнения фактического вывода с ожидаемым.

6.6.2. Порядок выполнения

1. Запустить 3 узла с конфигурационными файлами `node1.yaml`, `node2.yaml`, `node3.yaml`.
2. Дождаться строки "node ready" в логах каждого узла.
3. Последовательно выполнить команды согласно Приложению 2, зафиксировав вывод каждой команды.

6.6.3. Критерии успешности

Таблица 6.4 — Критерии успешности тестирования CLI

Команда	Ожидаемый результат
<code>cluster describe</code>	3 узла: 1 LEADER, 2 FOLLOWER
<code>topic create</code>	Выход с кодом 0; повторная попытка — сообщение <code>AlreadyExists</code>
<code>topic list</code>	Список содержит созданную тему
<code>topic describe</code>	Число партиций и RF соответствуют параметрам создания

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Команда	Ожидаемый результат
topic delete	Выход с кодом 0; topic describe — NOT_FOUND
produce --acks 0	Выход с кодом 0 немедленно
produce --acks all	Выход с кодом 0 после подтверждения кворума
consume --count 10	Вывод 10 строк с payload и метаданными смещений
offset earliest	Значение смещения первого доступного пакета
offset latest	Значение следующего назначаемого смещения
offset by-time	Смещение первого пакета с $base_timestamp \geq timestamp$

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1

КОНФИГУРАЦИЯ ТЕСТОВОГО КЛАСТЕРА

Для проведения интеграционных тестов используется кластер из 3 узлов, развёрнутый на одной машине. Ниже приведены конфигурационные файлы для каждого узла.

Конфигурация узла 1 (node1.yaml):

```
node_id: 1
raft_address: "127.0.0.1:8081"

peers:
  - id: 2
    address: "127.0.0.1:8082"
  - id: 3
    address: "127.0.0.1:8083"

data_dir: "/tmp/bunnymq-test/node1"

grpc:
  data_port: 9090
  management_port: 9091
  metrics_port: 9092

auth:
  tokens: []

log:
  level: "info"

defaults:
  segment_max_bytes: 1048576 # 1 МиБ (уменьшено для тестирования
  роллинга)
  retention_ms: 3600000 # 1 час
  retention_bytes: -1
```

Конфигурации узлов 2 и 3 аналогичны с заменой node_id (2 и 3), raft_address (127.0.0.1:8082 / 127.0.0.1:8083) и портов gRPC (9093/9094/9095 и 9096/9097/9098 соответственно).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Запуск кластера:

```
./bunnymq --config node1.yaml &  
./bunnymq --config node2.yaml &  
./bunnymq --config node3.yaml &
```

Проверка готовности:

```
./bunnymq-cli cluster describe --addr 127.0.0.1:9091  
# Ожидаемый вывод:  
# Cluster: 3 nodes  
#   Node 1  127.0.0.1:9090  LEADER    (metadata)  
#   Node 2  127.0.0.1:9093  FOLLOWER  
#   Node 3  127.0.0.1:9096  FOLLOWER
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2**ПЕРЕЧЕНЬ КОМАНД `bunnymq-cli` ДЛЯ ФУНКЦИОНАЛЬНОГО ТЕСТИРОВАНИЯ**

Ниже приведён перечень команд `bunnymq-cli`, проверяемых в ходе функционального тестирования. Все команды выполняются против узла 1 (`--addr 127.0.0.1:9091` для Management, `--addr 127.0.0.1:9090` для Data).

Управление кластером:

```
./bunnymq-cli cluster describe --addr 127.0.0.1:9091
```

Управление темами:

```
# Создание темы с 3 партициями и RF=3
./bunnymq-cli topic create testTopic --partitions 3 --replication-
factor 3 \
  --addr 127.0.0.1:9091

# Список тем
./bunnymq-cli topic list --addr 127.0.0.1:9091

# Описание темы
./bunnymq-cli topic describe testTopic --addr 127.0.0.1:9091

# Изменение числа партиций
./bunnymq-cli topic alter-partitions testTopic --new-count 6 --addr
127.0.0.1:9091

# Изменение параметров хранения (7 суток)
./bunnymq-cli topic alter-retention testTopic --retention-ms
604800000 \
  --addr 127.0.0.1:9091

# Удаление темы
./bunnymq-cli topic delete testTopic --addr 127.0.0.1:9091
```

Публикация и чтение:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# Публикация с acks=0 (без ожидания кворума)
./bunmq-cli produce --topic testTopic --partition 0 --acks 0 \
  --payload "test message 1" --addr 127.0.0.1:9090

# Публикация с acks=all (с ожиданием кворума)
./bunmq-cli produce --topic testTopic --partition 0 --acks all \
  --payload "test message 2" --addr 127.0.0.1:9090

# Чтение 10 сообщений начиная со смещения 0
./bunmq-cli consume --topic testTopic --partition 0 \
  --offset 0 --count 10 --addr 127.0.0.1:9090

# Чтение с долгим опросом (ожидание до 5 с)
./bunmq-cli consume --topic testTopic --partition 0 \
  --offset 0 --count 1 --max-wait 5000 --addr 127.0.0.1:9090
```

Запросы смещений:

```
./bunmq-cli offset earliest --topic testTopic --partition 0 \
  --addr 127.0.0.1:9090

./bunmq-cli offset latest --topic testTopic --partition 0 \
  --addr 127.0.0.1:9090

./bunmq-cli offset by-time --topic testTopic --partition 0 \
  --timestamp 1714000000000 --addr 127.0.0.1:9090
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.301–79 Программа и методика испытаний. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).
10. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
11. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]